

Documentation Technique

Application de scan alimentaire NutriScan

Mathieu Zavazzo

Table des matières :

Introduction Technique	3
Stack Technique	3
Environnement de développement	3
Tableau Récapitulatif	4
Architecture	5
Structure du projet	5
Base de données	6
Schéma Relationnel	6
Schéma de Base de Données	7
Tests JUnit	8
Cas testés	8
Exemples de tests	8
Mise en place du projet	10
Prérequis	10
Etapes	10

Introduction Technique

NutriScan est une application mobile Android permettant la gestion et l'analyse de produits alimentaires au sein d'entreprises, de cantines ou pour un usage personnel. Elle permet le scan de codes-barres, le tri par allergènes et catégories, la gestion de favoris, et l'affichage des informations nutritionnelles détaillées incluant le calcul du Nutri-Score.

L'application repose sur l'API publique OpenFoodFacts pour la récupération des données produits, et sur une base de données locale SQLite via Room pour la persistance hors ligne.

Stack Technique

Environnement de développement

- **IDE :** Android Studio est utilisé comme environnement de développement intégré principal
- **Langage :** Kotlin 2.3.20
- **SDK minimum :** API 28 Android 9 (Pie), couvrant ~95% des appareils actifs
- **SDK cible :** API 36 — Android 16

- **Git**
- **Gradle** : 8.13
- **JUnit** : 4

Tableau Récapitulatif

Outil / Technologie	Rôle dans le projet
Kotlin 2.3.20	Langage principal de développement natif pour Android
Room 2.8.4	Base de données SQLite locale
Jetpack Compose 2026.03.00	Framework UI moderne remplaçant les vues XML traditionnelles
Gradle 8.13	Système de build gérant les dépendances, la compilation et la génération de l'APK
Android Studio	Environnement de développement intégré principal pour Android
Retrofit 2.11.0	Client HTTP typé facilitant les appels vers l'API OpenFoodFacts et la désérialisation des réponses JSON
ML Kit Barcode 17.3.0	Bibliothèque Google permettant la détection et la lecture des codes-barres via la caméra

Kotlinx Coroutines Flow 1.10.2	Gestion des flux de données asynchrones et réactifs entre la base de données, le ViewModel et l'interface
Lifecycle / ViewModel 2.10.0	Survie des données UI aux rotations d'écran et gestion du cycle de vie des composants
JUnit 4	Framework de tests

Architecture

Structure du projet

L'application NutriScan adopte le pattern MVVM, recommandé par Google, pour assurer une séparation stricte entre la logique métier et l'interface utilisateur. Cette structure facilite la maintenance et la réactivité de l'application.

UI (Compose) ↔ ViewModel ↔ Repository ↔ Data (Room / Retrofit)

data/local/ : Contient la persistance SQLite avec Room (entités, dao, appdatabase)

data/remote/ : Gère la communication avec l'API OpenFoodFacts via Retrofit

data/repository/ : L'implémentation concrète qui fait le pont entre le local et le distant

data/domain/model/ : Contient les objets "métier"

data/domain/repository/ (Interface) : Définit le contrat (ex: getProduct(id)), sans dire comment le faire

Couche UI entièrement développée à partir du framework Jetpack Compose :

ui/components/ : Éléments graphiques réutilisables

ui/screens/ : Écrans principaux de l'application

ui/theme/ : Thème global

ui/navbar/ : Gestion de la navigation principale de l'application.

viewmodel/ProductViewModel.kt : Gère l'état de l'interface (liste des produits, chargement, erreurs). Il survit aux rotations d'écran

viewmodel/ProductViewModelFactory.kt : Permet l'injection du Repository dans le ViewModel

Base de données

Schéma Relationnel

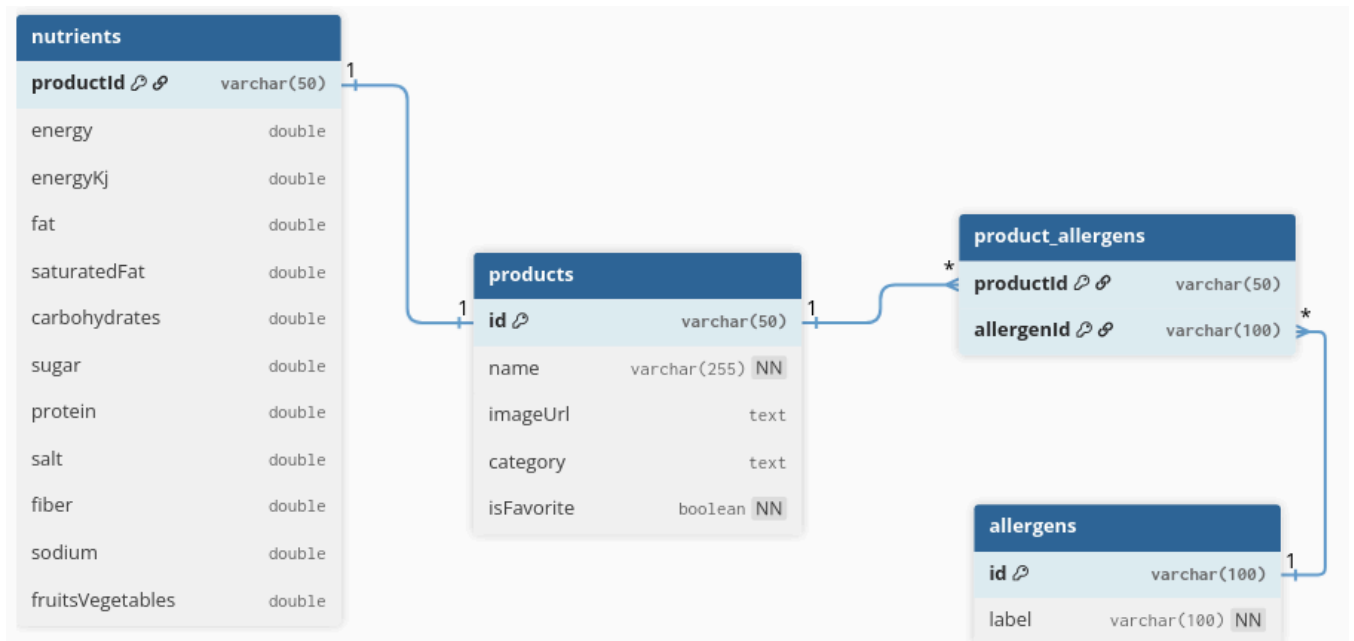
```
nutrients (productId, energy, energyKj, fat, saturatedFat,
carbohydrates, sugar, protein, salt, fiber, sodium,
fruitsVegetables)
Clé primaire : productId
Clé étrangère : productId en référence à id de products

products (id, name, imageUrl, category, isFavorite)
Clé primaire : id

allergens (id, label)
Clé primaire : id

product_allergens (productId, allergenId)
Clé primaire : productId, allergenId
Clé étrangère productId en référence à id de products,
allergenId en référence à id de allergens
```

Schéma de Base de Données



Tests JUnit

Cas testés

- Test unitaire sur le calcul du nutri score
- Test fonctionnel sur l'insertion, la suppression et la mise en favori d'un produit

Exemples de tests

Test sur le calcul du nutri score E :

```
class NutriScoreUnitTest {  
  
    private fun nutriments(  
        energy: Double = 0.0,  
        energyKj: Double = 0.0,  
        fat: Double = 0.0,  
        saturatedFat: Double = 0.0,  
        carbohydrates: Double = 0.0,  
        sugar: Double = 0.0,  
        protein: Double = 0.0,  
        salt: Double = 0.0,  
        fiber: Double = 0.0,  
        sodium: Double = 0.0,  
        fruitsVegetables: Double = 0.0  
    ) = Nutriments(energy, energyKj, fat, saturatedFat,  
        carbohydrates, sugar, protein, salt, fiber, sodium,  
        fruitsVegetables)  
  
    @Test  
    fun `produit tres sucre doit etre E`() {  
        val n = nutriments(energyKj = 3500.0, sugar = 60.0,  
            saturatedFat = 10.0, salt = 2.0)  
        assertEquals(NutriScoreLetter.E, computeNutriScore(n,  
            null).letter)  
    }  
}
```

Test sur l'ajout d'un produit :

```
class ProductDaoInstrumentedTest {  
  
    private lateinit var db: AppDatabase  
    private lateinit var dao: ProductDao  
  
    @Before  
    fun setup() {  
        val context =
```



```
ApplicationProvider.getApplicationContext<Context>()
    db = Room.inMemoryDatabaseBuilder(context,
AppDatabase::class.java)
        .allowMainThreadQueries()
        .build()
    dao = db.productDao()
}

@After
fun teardown() {
    db.close()
}

@Test
fun inserterEtRecupererUnProduit() = runBlocking {
    val product = ProductEntity(
        id = "3017620422003",
        name = "Nutella",
        imageUrl = null,
        category = "Pâtes à tartiner",
        isFavorite = false
    )
    dao.insertProduct(product)
    val result = dao.getProductWithDetails("3017620422003")
    assertNotNull(result)
    assertEquals("Nutella", result?.product?.name)
}
}
```

Mise en place du projet

Prérequis

- Android Studio
- JDK 11 ou supérieur (JDK 17 par défaut avec Android Studio)
- Git
- Appareil Android SDK 28 (Android 9.0) minimum ou émulateur

Etapes

- Ouverture du projet dans Android Studio (clone du dépôt GitHub ou ouverture du fichier ZIP après décompression)
- Synchroniser le projet avec Gradle
- Vérifier le SDK et JDK
- Lancer le projet sur l'appareil ou sur l'émulateur